

Functional Programming

Radu Răzvan Slăvescu

Technical University of Cluj-Napoca
Department of Computer Science

Outline

Functional Programming

- Definition of Functional Programming

- Where is FP used?

- Brief history

Administrativa

- Course overview

- Bibliography

- Evaluation

Defining functions

if-then-else and Pattern Matching

Recursion

- Looping via recursion

- Tail recursion

Administrativia

Lectures

Assoc. prof. dr. eng. Radu Răzvan Slăvescu

Course Materials

1. Lecture slides (in English) + lab documents posted on moodle every week
2. Enrollment key provided by the TA
You MUST use the key of your group!
3. Questions: via moodle messaging

Definition

Functional Programming (FP) = a style of programming which employs **functions** applied to arguments as basic blocks for building programs. The arguments themselves can be function calls, so complex programs are built via **function composition**.

Observation

The functional paradigm belongs to **declarative** programming: you specify **what** the program does, not **how** it does it.

Definition

Program = Function

Principles of (pure) Functional Paradigm

- ▶ Functions are "first class citizens" (can be passed as arguments, returned by other functions, stored in variables)
- ▶ Functions have **no side effects** (or highly controlled side effects)
- ▶ **Expressions** (= get evaluated to some value), not statements (= perform some action)
- ▶ No sequential execution of code, just combining functions
- ▶ **Immutable** variables: their value does not change over time
- ▶ There is no "state of the program"
- ▶ There are no loops, just recursive calls

What is FP?

Features of the FP languages we will study

- ▶ *strongly typed*: every entity has a type attached
- ▶ *statically typed*: type detected at compile time
- ▶ perform *type inference*: each entity's type is automatically deduced

Language vs. implementation

Language: Haskell. Implementation: `ghc` (software toolkit)

Why FP?

Modern software challenges...

1. complexity increases
2. time of development should decrease
3. number of errors should decrease

...and solutions offered by Functional Programming

1. supports writing highly abstracted, clear and concise code
2. allows software reuse and rapid prototyping
3. permits reasoning about the program's correctness
4. permits easy parallelization of the code

Where if FP used?

Functional Programming - some application domains

1. Erlang: WhatsApp (50 engineers, 900M users)
2. Haskell: IOG (Cardano - third-gen blockchain platform)
3. Haskell: Facebook (the Sigma anti-spam system)
4. Artificial Intelligence tools and languages:
 - ▶ LEAN, an FP language and a proof assistant
 - ▶ JAX, a Python library for numerical computing
 - ▶ Grammatical Framework, a tool for NLP
 - ▶ CLIPS, a rule based expert system shell

A little bit of history

Lambda calculus: 1930s

- ▶ Lambda calculus - a theory by Alonzo Church about what functions are able to compute
- ▶ could be seen as a minimalist programming language
- ▶ virtually every functional language is based on this theory
- ▶ useful in many fields, e.g. expressing semantics of text

A little bit of history

LISP (LISt Processing): 1950s, John McCarthy

- ▶ functional language with ideas from lambda calculus
- ▶ basic blocks: atoms, lists, forms
- ▶ allows variable change (unfortunately!)
- ▶ nowadays dialects:
 1. Common LISP
 2. Scheme
 3. Racket
 4. Clojure (integrated with Java platform)
 5. Hy (integrated with Python)
- ▶ implementations
 1. gcl (GNU Common LISP)
 2. CLISP
 3. Steel Bank Common LISP
 4. Allegro CL

A little bit of history

ML (Meta Language): 1970s, Robin Milner

- ▶ featuring polymorphic types and type inference
- ▶ implementations:
 1. sml (Standard ML)
 2. polyML
 3. Caml
 4. Objective Caml

A little bit of history

Miranda family

Miranda means "Admirable"

A little bit of history

Miranda family: Haskell (after Haskell Curry): 1987

- ▶ purely functional, with lazy evaluation
- ▶ standard: Haskell 2010 report (succeeding Haskell 98)
- ▶ implementations:
 1. ghc (Glasgow Haskell Compiler)
 2. hugs (Haskell User's Gofer System)

Multiparadigm programming languages featuring FP

Multiparadigm languages

Scala	immutability, laziness. Compiles to bytecode
Rust	expressions, types, concurrency
F# (for .NET)	OO + FP (Caml-like)
C#	immutability, function composition
Python	lambdas, higher-order functions, lazy objects

Objectives

After completing this course you should be able to:

- ▶ write idiomatic, efficient functional code
- ▶ formally prove properties of functional programs
- ▶ develop a basic understanding of Lambda calculus

Lectures overview

Basics of programming in Haskell

- ▶ primitive data types
- ▶ functions and recursion
- ▶ evaluation (call-by-value, call-by-name, call-by-need)
- ▶ type inference
- ▶ lists and list comprehensions

Lectures overview

FP specific elements

- ▶ higher-order functions HOFs
(incl. anonymous functions, partial application)
- ▶ infinite data and lazy evaluation
- ▶ transformation and reasoning on programs
- ▶ elements of Lambda calculus

Lab overview

Lab materials (+ related lectures) must be read in advance

Elm lab: weeks 1 - 6

- ▶ basics of Elm
- ▶ defining Elm functions by recursion
- ▶ write small demo applications

Haskell: weeks 8 - 13

- ▶ practice writing Haskell code
- ▶ write small demo applications

Bibliography



R. R. Slăvescu

Functional Programming - lecture notes.

Technical University of Cluj-Napoca, 2026.



G. Hutton

Programming in Haskell, 2nd edition.

Cambridge University Press, 2016

slides available from the course web page



M. Lipovača.

Learn you a Haskell for Great Good.

<http://learnyouahaskell.com>

Bibliography



Haskell wiki page

`www.haskell.org`



Phillip Hagenlocher.

Learn Haskell by Example.

Manning, 2024

Evaluation

How is your mark computed?

Weighted average:

- ▶ Final examination: 50% (written/moodle)
- ▶ Lab activity 50%
 - ▶ week 7 (Elm): Lab test 1: 25%
 - ▶ week 14 (Haskell): Lab test 2: 25%

Evaluation

Bonuses:

- ▶ Challenges
- ▶ Kahoot Yellow Jersey competition (1 point per competition winner + 1.5, 1.4, ... , 0.6 for the global standings)
If Kahoot is available
- ▶ outstanding activity during lectures/labs

Condition for passing

Lab: $(0.5*L1 + 0.5*L2) \geq 5$

Exam: $(0.25*L1 + 0.25*L2 + 0.25*E1 + 0.25*E2) + \text{bonus} \geq 5.00$

Final Exam

Exam

- ▶ 50 % of the final mark
- ▶ quick questions + problem
- ▶ If failed: repeated during the next session of exams

A couple of small Haskell examples

Computing reciprocal ($\rightarrow$ partial application)

```
(1/)
```

Sorting lists ($\rightarrow$ list comprehension + HOF)

```
q []      = []
q (x:xs) = q [s | s <- xs, s<=x] ++
           [x] ++
           q (filter (>x) xs)
```

Generating primes ($\rightarrow$ infinite lists)

```
pr (p:ns) = p:pr [ n | n <- ns, n `mod` p > 0]
```

Styles of interacting with computer

REPL versus batch

1. **Read-Eval-Print Loop (REPL)**: enter functions from the keyboard and get answers from the system (conversational)
2. file: functions stored in a file and loaded from there (batch)

How do we define functions?

Definition (Function from domain D to codomain C)

Total/partial: defined for *every* element / *some* elements in D

Example (Haskell, batch)

```
area :: Double -> Double
area r = 3.14 * r * r
```

Example (Haskell, REPL)

```
Prelude> area r = 3.14 * r * r
Prelude> area 3
28.259999999999998
```

How do we know the type of a function?

Example (square of an integer: `sq`)

```
Prelude> sq :: Int -> Int; sq x = x * x
```

```
Prelude> :type sq  
sq :: Int -> Int
```

Example (Most general solution for `sq` type)

```
Prelude> sq x = x*x
```

```
Prelude> :type sq  
sq :: Num a => a -> a
```

How do we call a function?

Calling functions

Math: $f(x, y * z, g(t))$

FP: $f\ x\ (y * z)\ (g\ t)$

Note

1. as a rule, no parentheses separate the function name and its arguments;
2. pay attention to the precedence: $\text{fact } n - 1$ versus $\text{fact } (n-1)$; if not sure, use parentheses

if expression

if-then-else

if E then ET else EF:

1. E gets evaluated, then ...
2. either ET or EF is evaluated, but never both of them
3. else is **mandatory** (Why?)

Pattern matching mechanism

Example (Product of elements in a list)

```
prod1 [] = 1 --equation 1  
prod1 (n:ns) = n * prod1 ns --equation 2
```

Observations

1. each equation defines a pattern, telling what the function returns when its argument looks in a specific way ($\rightarrow$ declarative style)
2. pattern order **does** matter
3. `n:ns` matched on `[1, 2, 3]` binds `n` to 1 and `ns` to `[2, 3]`
4. `1:ns` matched on `[1, 2, 3]` binds `ns` to `[2, 3]`;
5. `1:ns` fails when attempting to match `[2, 3]`
6. `n:_:ns` matched on `[1, 2, 3]` binds `n` to 1 and `ns` to `[3]`

Pattern matching mechanism

Example (Product of elements in a list using case)

```
prod1 xs = case xs of
              []      -> 1
              (n:ns)  -> n * prod1 ns
```

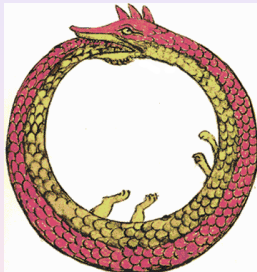
Example (Product of elements in a list: no patterns)

```
prod1 ns = if null ns then 1
           else (head ns) * prod1 (tail ns)
```

Observations

1. using patterns increases code clarity ...
2. ... and allows a better analysis of cases, thus leading to a better performance of the generated code

Recursion: a function calls itself in its body



Ouroboros snake (with permission from Wikimedia Commons)

Recursion: main mechanism to implement loops

Definition

Linear recursive function It has only one recursive call to itself each time it runs.

Example (Greatest common divisor: Euclid's Algorithm)

```
euclidgcd m n = if (m == 0) then n
                else euclidgcd (n `mod` m) m
```

Example (Power, linear)

```
power m n = if (m==0) then 0 -- what if n==0
            else if n==0 then 1
                else m * (power m (n-1))
```

Recursive functions

Example (Fibonacci $F_0 = 0$, $F_1 = 1$, $F_n = F_{n-1} + F_{n-2}$)

```
fib n = if n==0 then 0
        else if n==1 then 1
            else fib (n-1) + fib (n-2)
```

Example (Ackermann-Péter function)

```
ack m n = if m==0 then (n+1)
            else if n==0 then ack (m-1) 1
                else ack (m-1) (ack m (n-1))
```

Remark

The number of recursive calls of this function increases extremely rapidly with its arguments.

Recursive functions

Definition

A function is **tail recursive** if it returns either something computed directly or something returned by its recursive call (the last thing it does is to call itself)

Example (A tail recursive function)

```
iseven n =  
  if (n==0) then True  
  else if (n==1) then False  
    else iseven (n-2)
```

Recursive functions

Example (factorial - a not tail recursive version)

```
fact n = if (n == 0) then 1  
         else n * fact (n-1)
```

Example (factorial - a tail recursive version)

```
factacc n a = if (n==0) then a  
              else factacc (n-1) (n*a)
```

fact: successive calls waste stack space and time

Example

```
fact 3
= 3 * fact 2
= 3 * (2 * fact 1)
= 3 * (2 * (1 * fact 0))
= 3 * (2 * (1 * 1))
= 3 * (2 * 1)
= 3 * 2
= 6
```

fact 0

↓1

fact 1

↓1

fact 2

↓2

factacc: no space and time wasted

Example

factacc 3 1 =

factacc (3-1) (3*1) =

factacc (2-1) (2*3) =

factacc (1-1) (1*6) =

6

factacc 0 6

6

factacc 1 6

factacc 2 3

Remark

Each call can overwrite the stack frame of the previous one, hence no need for extra space or stack processing time

factacc: no space and time wasted

Example

factacc 3 1 =

6

factacc (3-1) (3*1) =

factacc (2-1) (2*3) =

factacc (1-1) (1*6) =

6

Remarks

1. a tail recursive function is equivalent to an iteration and can automatically be re-written this way by the compiler
2. writing recursive functions in this manner helps saving stack **space (constant!)** and **time (stack manipulation)**

Don't Repeat Yourself: Local declarations in Haskell

Example (Top-down approach: **where**)

```
f r = sq ((x * y) / pi)
  where
    sq z = z ** 0.5 -- square root
    pi = 3.14
    (x, y) = r -- deconstruct a tuple
```

Notes

1. don't forget that in Haskell **indentation is important**.
2. Local declarations enter the scope in the same time

Don't Repeat Yourself: Local declarations in Haskell

Example (Bottom-up approach: **let ... in ...**)

```
f x = let
    sq z = z ** 0.5 -- square root
    pi = 3.14
    (x, y) = r -- deconstruct a tuple
  in
    sq ((x * y) / pi)
```

```
Prelude> f (16, 314)
40.0
```

Mutually Recursive Functions

Haskell: define either before or after use

Example (Order not important)

```
isevn n = if (n==0) then True else isodd (n-1)
isodd n = if (n==0) then False else isevn (n-1)
```

or

Example (Order not important)

```
isodd n = if (n==0) then False else isevn (n-1)
isevn n = if (n==0) then True else isodd (n-1)
```

Summary

- ▶ FP is a style of programming which employs functions applied to arguments as basic blocks for building programs
- ▶ We use functions and functions alone
- ▶ No side effects. No variable change. No program state. No sequences of instructions. No statements. No loops.
- ▶ Many functional languages/implementations available; among them, Haskell/ghc and Elm
- ▶ Pattern Matching is used to express alternatives.
- ▶ Recursion and function composition are employed for writing complex functional programs
- ▶ Tail recursion improves performance

Further reading

Learn more

LYHFGG: Chapter 1, Chapter 2 up to "Texas Ranges"

Programming in Haskell: Chapter 1 and Chapter 2

Why haskell in Cardano?

<https://www.youtube.com/watch?v=1HGUu-EIb5g>

Functional Programming Applications - further info

1. galois.com (critical systems)
2. commercial users of FP: <http://cufp.org>
3. <https://www.lightbend.com/case-studies>
4. projects developed in LISP: www.franz.com/success